```python
# --- Step 1 -----------------------------------------------------
# Find runs of consecutive CpGs where the same parental hap is the
# meth outlier, was transmitted to the kid, and the kid hap on that
# side has ~the same meth level.
TOL = 0.10
df_meth_hits = (df_meth
    # sites where exactly one parental hap is the meth outlier
    .with_columns(
        n_hi         = ((pl.col("dad_A") > 0.5)
                      + (pl.col("dad_B") > 0.5)
                      + (pl.col("mom_C") > 0.5)
                      + (pl.col("mom_D") > 0.5)),
        outlier_hap  = pl.when(pl.col("dad_A") > 0.5).then(pl.lit("A"))
                        .when(pl.col("dad_B") > 0.5).then(pl.lit("B"))
                        .when(pl.col("mom_C") > 0.5).then(pl.lit("C"))
                        .otherwise(pl.lit("D")),
        outlier_meth = pl.max_horizontal("dad_A", "dad_B", "mom_C", "mom_D"),
    )
    .filter(pl.col("n_hi") == 1)
    # group consecutive CpGs sharing the same outlier hap
    .with_columns(
        run_id = (pl.col("outlier_hap")
                  != pl.col("outlier_hap").shift(1)).cum_sum()
    )
    # collapse each run; outlier_meth = mean over its CpGs
    .group_by("run_id", maintain_order=True)
    .agg(
        chrom        = pl.col("chrom").first(),
        start        = pl.col("start").min(),
        end          = pl.col("start").max(),
        n            = pl.len(),
        outlier_hap  = pl.col("outlier_hap").first(),
        outlier_meth = pl.col("outlier_meth").mean(),
        pat_hap      = pl.col("pat_hap").first(),
        mat_hap      = pl.col("mat_hap").first(),
        kid_pat      = pl.col("kid_pat").mean(),
        kid_mat      = pl.col("kid_mat").mean(),
    )
    .filter(pl.col("n") >= 3)
    # kid inherits the outlier hap with ~unchanged meth
    .filter(
        ((pl.col("outlier_hap") == pl.col("pat_hap"))
         & ((pl.col("kid_pat") - pl.col("outlier_meth")).abs() < TOL))
        | ((pl.col("outlier_hap") == pl.col("mat_hap"))
           & ((pl.col("kid_mat") - pl.col("outlier_meth")).abs() < TOL))
    )
)

# --- Step 2 -----------------------------------------------------
# Exactly one parental hap carries the ALT; kid inherits it on the
# matching side.
df_geno_hits = (df_geno
    # tag each SNV with the ALT count and the (unique) outlier hap
    .with_columns(
        n_alt       = pl.sum_horizontal("dad_A", "dad_B", "mom_C", "mom_D"),
        outlier_hap = pl.when(pl.col("dad_A") == 1).then(pl.lit("A"))
                       .when(pl.col("dad_B") == 1).then(pl.lit("B"))
                       .when(pl.col("mom_C") == 1).then(pl.lit("C"))
                       .otherwise(pl.lit("D")),
    )
    # keep SNVs with a single ALT-carrying parental hap
    .filter(pl.col("n_alt") == 1)
    # kid inherits that ALT on the matching side
    .filter(
        (pl.col("outlier_hap").is_in(["A", "B"]) & (pl.col("kid_pat") == 1))
        | (pl.col("outlier_hap").is_in(["C", "D"]) & (pl.col("kid_mat") == 1))
    )
)

# --- Step 3 -----------------------------------------------------
# Compound het = meth-outlier and geno-outlier from different
# parents, same locus.
df_compound_het = (df_meth_hits
    .rename({"outlier_hap": "outlier_hap_meth"})
    # pair each meth-outlier region with the nearest geno-outlier
    # SNV on the same chromosome, within 500 bp
    .join_asof(
        df_geno_hits.rename({"outlier_hap": "outlier_hap_geno"}),
        on="start", by="chrom",
        strategy="nearest", tolerance=500,
    )
    # keep loci where the meth and geno outliers are in trans
    # (i.e., come from different parents)
    .filter(
        (pl.col("outlier_hap_meth").is_in(["A", "B"])
         & pl.col("outlier_hap_geno").is_in(["C", "D"]))
        | (pl.col("outlier_hap_meth").is_in(["C", "D"])
           & pl.col("outlier_hap_geno").is_in(["A", "B"]))
    )
)
```